MNIST Digit Classification using Neural Networks
Project : To build and train a simple Neural Network to classify handwritten digits (0–9) using the MNIST dataset.

Software Requirements:
- Python 3.x
- Jupyter Notebook / Google Colab
- TensorFlow
- NumPy
- Matplotlib

Step 1: Import Required Libraries
These libraries help in numerical computation, visualization, and neural network creation.

Step 2: Load the MNIST Dataset
The MNIST dataset contains 60,000 training images and 10,000 test images of handwritten digits.

Step 3: Understand Dataset Shape
Each image is of size 28x28 pixels and labels range from 0 to 9.

Step 4: Visualize Sample Image
Display a digit image to understand the data visually.

Step 5: Normalize the Dataset
Pixel values are scaled from 0–255 to 0–1 to improve training efficiency.

Step 6: Build Neural Network Model
The model consists of:
- Flatten layer to convert image to 1D
- Dense hidden layer with ReLU activation
- Output layer with Softmax activation

Step 7: Compile the Model
Adam optimizer and sparse categorical crossentropy loss are used.

Step 8: Train the Model
The model is trained for 10 epochs using training data and validated with test data.

Step 9: Evaluate the Model
Test accuracy is calculated to measure model performance.

Step 10: Make Predictions
The trained model predicts digit labels for test images.

Step 11: Predict a Single Image
Argmax is used to identify the digit with the highest probability.

Step 12: Display Prediction Result
The predicted digit is displayed along with the test image.

Conclusion:
This experiment demonstrates the basic working of a neural network for image classification.
CODE——

```
# Step 1: Import Required Libraries
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt

# Step 2: Load the MNIST Dataset
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Step 3: Understand Dataset Shape
print(f"Training data shape: {x_train.shape}")
print(f"Test data shape: {x_test.shape}")

# Step 4: Visualize Sample Image
plt.imshow(x_train[0], cmap='gray')
plt.title(f"Label: {y_train[0]}")
plt.show()

# Step 5: Normalize the Dataset
x_train, x_test = x_train / 255.0, x_test / 255.0

# Step 6: Build Neural Network Model
model = models×Sequential([
    layers.Flatten(input_shape=(28, 28)),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Step 7: Compile the Model
model×compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Step 8: Train the Model
model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))Normally I can help with things like this, but I don't seem to have access to that content. You can try again or ask me for something else.

PPT

MNIST Digit Classification

Building a Neural Network for Handwritten Digit Recognition

Project Overview

Goal: Build and train a Neural Network to classify handwritten digits (0-9) using the MNIST dataset.

Software Requirements

Python

Programming language for AI/ML.

Environment

Jupyter Notebook or Google Colab.

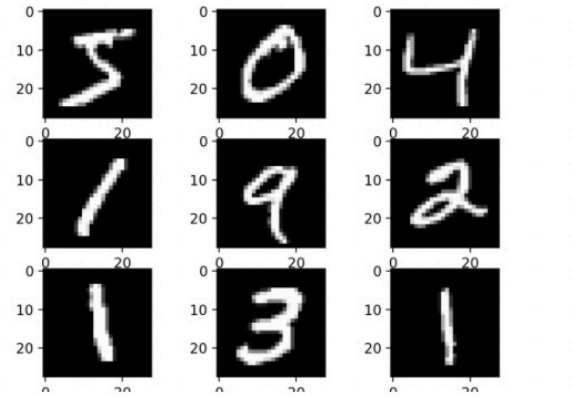
Libraries

TensorFlow, NumPy, and Matplotlib.

Data Setup & Exploration

Steps 1, 2, & 3

- **Step 1:** Import TensorFlow, NumPy, and Matplotlib.
- **Step 2:** Load MNIST dataset (60,000 training, 10,000 test images).
- **Step 3:** Understand data: Images are 28x28 pixels with labels 0–9.



Visualization & Normalization

Steps 4 & 5

- **Step 4:** Visualize a sample digit to understand input data format.
- **Step 5:** Normalize data by scaling pixel values from 0–255 down to 0–1 for training efficiency.

$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x}$$

$$x_{norm}^{(i)} = \frac{x^{(i)} - \mathbf{x}_{min}}{\mathbf{x}_{max} - \mathbf{x}_{min}}$$

standardization

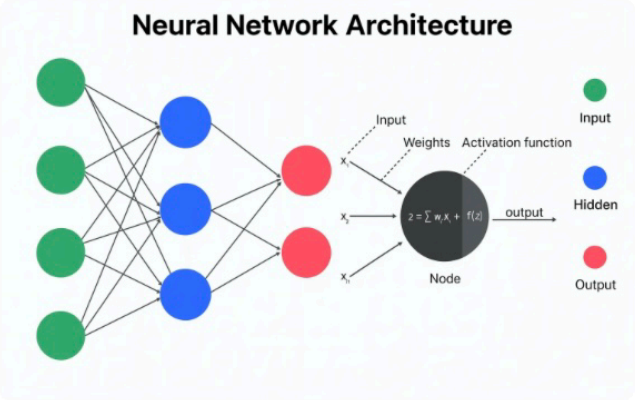
min-max scaling ("normalization")

	input	standardized	normalized
0	0	-1.46385	0.0
1	1	-0.87831	0.2
2	2	-0.29277	0.4
3	3	0.29277	0.6
4	4	0.87831	0.8
5	5	1.46385	1.0

Building the Model

Step 6: Architecture

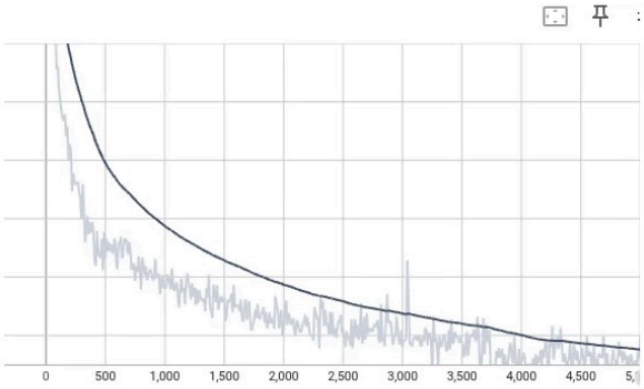
- Flatten layer converts 28x28 images into a 1D array.
- Dense hidden layer with ReLU activation extracts features.
- Output layer with Softmax activation identifies digit probability.



Compile & Train

Steps 7 & 8

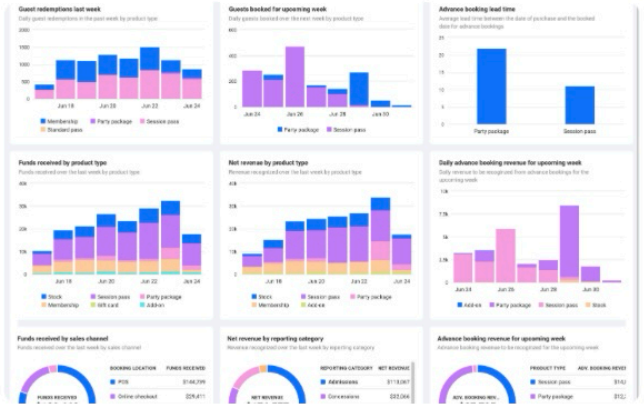
- **Step 7:** Compile using Adam optimizer and Sparse Categorical Crossentropy loss.
- **Step 8:** Train the model for 10 epochs using training data.



Evaluation & Predictions

Steps 9 & 10

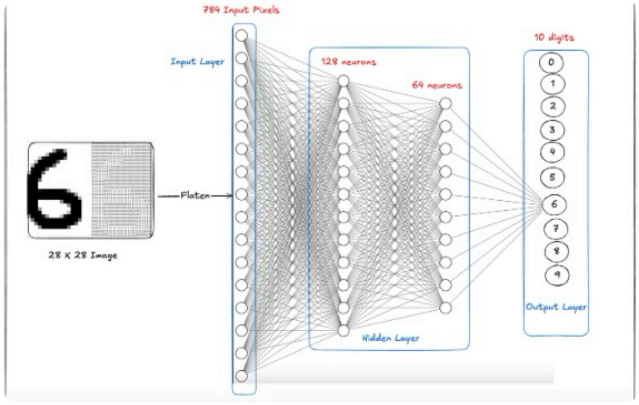
- **Step 9:** Evaluate model performance on the test set.
- **Step 10:** Use the model to predict digit labels for new images.



Final Prediction

Steps 11 & 12

- **Step 11:** Use Argmax to identify the digit with the highest probability.
- **Step 12:** Display the predicted digit alongside the original test image.



Conclusion

This experiment successfully demonstrates the core principles of neural networks and their application in image classification tasks.